OHS REMOTE — CAPSTONE DOCUMENTATION

Capstone347 • CSD-235

Backend Developer Setup Guide

Environment, installation, and day-to-day operation of the OHS Remote FastAPI backend

Capstone347 — CSD-235

April 14, 2026

Contents

1 Scope **2**

2 Environment Requirements **2**
- 2.1 Software to install on your host machine . . . 2
- 2.2 Versions pinned inside the stack . . . 2
- 2.3 Accounts you need to create . . . 3

3 Installation **3**
- 3.1 Step 1 — Clone the repository and prepare environment files . . . 3
- 3.2 Step 2 — Generate a secret key . . . 3
- 3.3 Step 3 — SMTP (Gmail App Password) . . . 4
- 3.4 Step 4 — ngrok (webhook tunnel) . . . 4
- 3.5 Step 5 — Stripe . . . 5
- 3.6 Step 6 — OpenAI API key . . . 6

4 Configuration Reference **6**

5 Running the Stack **7**
- 5.1 First run . . . 7
- 5.2 Seed the plans table . . . 7
- 5.3 Verification checklist . . . 7

6 Day-to-day Commands **8**
- 6.1 Lifecycle . . . 8
- 6.2 Logs . . . 8
- 6.3 Tests, linting, type checking . . . 8
- 6.4 Database migrations . . . 8
- 6.5 Adding a Python dependency . . . 9

7 Build and Deployment **9**

8 Troubleshooting **9**
- 8.1 App refuses to start with a Pydantic `ValidationError` . . . 9
- 8.2 Port already in use . . . 10
- 8.3 "Can't connect to MySQL server on 'mysql'" . . . 10
- 8.4 Alembic "Target database is not up to date" or multiple heads . . . 10
- 8.5 Gmail rejects SMTP authentication . . . 11
- 8.6 Stripe webhook never fires . . . 11
- 8.7 OpenAI / SJP generation errors . . . 11
- 8.8 Code changes not taking effect . . . 11
- 8.9 `ModuleNotFoundError` on imports . . . 12
- 8.10 File permission errors writing to `data/` . . . 12
- 8.11 Out of disk space . . . 12
- 8.12 Nothing works — start over . . . 12

9 Where to Go Next **12**

1 Scope

This document is the authoritative setup guide for the **OHS Remote backend**, a FastAPI service that handles order management, document generation, Stripe payments, email delivery, and AI-powered Safe Job Procedure (SJP) generation. It is written so that a developer with no prior exposure to this project can clone the repository and bring a working local environment online from scratch.

The frontend repository is documented separately in `developer_setup_frontend.tex`. Reading both guides in order is the recommended path for a new full-stack developer.

Important

Backend repository: https://github.com/Capstone347/ohs_backend

Important

Docker is the only supported development path for this project. The backend has a precise environment (MySQL version, Python version, system libraries for `python-docx` and the MySQL driver) and Docker Compose pins all of it. Running outside Docker is possible but unsupported. Every command in this guide assumes you are running the stack through `docker-compose`.

2 Environment Requirements

2.1 Software to install on your host machine

- **Docker Desktop** — https://www.docker.com/products/docker-desktop. The stack uses Docker Compose (bundled with Docker Desktop) to orchestrate MySQL, the FastAPI app, and an ngrok tunnel.
- **Git** — https://git-scm.com/downloads.

You do *not* need Python, MySQL, or any system libraries installed on your host. They all live inside the Docker containers.

2.2 Versions pinned inside the stack

Component	Version
Python	3.11 (`python:3.11-slim` base image)
FastAPI	0.104+
SQLAlchemy	2.0+
Pydantic	2.x
Alembic	1.12+
MySQL	8.0
`python-docx`	1.1+

2.3 Accounts you need to create

Before you can bring the stack up you will need accounts at the following services. Each one provides one or more values that you will paste into `.env.docker`.

Service	Used for	Signup
Google	Gmail App Password for sending email in development	https://myaccount.google.com
ngrok	Public tunnel so Stripe webhooks can reach your laptop	https://dashboard.ngrok.com/signup
Stripe	Payment processing (use Test mode — ask the project lead for an invite to the shared project)	https://dashboard.stripe.com
OpenAI	AI generation of Safe Job Procedure content	https://platform.openai.com

3 Installation

3.1 Step 1 — Clone the repository and prepare environment files

```
git clone https://github.com/Capstone347/ohs_backend.git
cd ohs_backend
cp .env.docker.example .env.docker
```

The project uses two env-file templates:

File	Used by
`.env.docker`	Docker containers (app + ngrok). **This is the one you fill in.**
`.env`	Only needed if you insist on running the app outside Docker, which is unsupported.

Edit `.env.docker` and fill in the values described in the next sections. The app refuses to start if any required variable is missing or left as a placeholder like `...`, because `app/config.py` declares every required field with `Field(...)` and no default.

3.2 Step 2 — Generate a secret key

`SECRET_KEY` is required. Generate a strong random value:

```
openssl rand -hex 32
```

Paste the output into `.env.docker`:

```
SECRET_KEY=<paste output here>
```

3.3 Step 3 — SMTP (Gmail App Password)

The app sends emails (order confirmations, OTP codes, document delivery) via SMTP. Gmail works well for development but rejects your regular account password — you must create an **App Password**.

1. Go to https://myaccount.google.com and open **Security**.
2. Enable **2-Step Verification** if it is not already enabled.
3. At the bottom of the 2-Step Verification page, click **App passwords**, or open https://myaccount.google.com/apppasswords directly.
4. Under "Select app" choose **Mail**, under "Select device" choose **Other** and type a label like `OHS Remote`.
5. Click **Generate** and copy the 16-character password.

Fill in `.env.docker`:

```
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your-gmail-address@gmail.com
SMTP_PASSWORD=xxxx xxxx xxxx xxxx
SMTP_FROM_EMAIL=your-gmail-address@gmail.com
SMTP_FROM_NAME=OHS Remote Dev
```

Note

`SMTP_FROM_EMAIL` must match `SMTP_USER` when using Gmail, otherwise Gmail rejects the send. Spaces in the 16-character App Password are optional — `xxxx xxxx xxxx xxxx` and `xxxxxxxxxxxxxxxx` are both accepted.

3.4 Step 4 — ngrok (webhook tunnel)

ngrok exposes your local app to the public internet, which is required for Stripe to deliver webhook events during development.

1. Sign up for a free account at https://dashboard.ngrok.com/signup.
2. Open **Your Authtoken**: https://dashboard.ngrok.com/get-started/your-authtoken.
3. Copy the token.

Fill in `.env.docker`:

```
NGROK_AUTHTOKEN=your_token_here
```

When `docker-compose up` runs, the ngrok container automatically creates a public HTTPS tunnel to `app:8000`. The public URL is visible at http://localhost:4040.

Warning

The public ngrok URL rotates every time Docker restarts. You will need to update the Stripe webhook endpoint to the new URL after each restart. Free ngrok accounts allow a single active tunnel at a time — close any other running ngrok sessions before starting the stack.

Add the current ngrok URL to `ALLOWED_ORIGINS` in `.env.docker` so the frontend can communicate through the tunnel:

```
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:5173,http://localhost:8080,https://abcd-1234-56e
```

3.5 Step 5 — Stripe

Ask the project lead for an invitation to the shared Stripe project. Once accepted:

1. Log in at https://dashboard.stripe.com.
2. Toggle **Test mode** on (top-right corner).
3. Go to **Developers** → **API keys**: https://dashboard.stripe.com/test/apikeys.
4. Copy the **Secret key** (`sk_test_...`) and **Publishable key** (`pk_test_...`).

Create a webhook endpoint

Stripe needs to deliver payment events to your machine. The webhook endpoint must point at your ngrok URL.

1. Go to **Developers** → **Webhooks**: https://dashboard.stripe.com/test/webhooks.
2. Click **Add endpoint**.
3. **Endpoint URL**: your current ngrok URL plus the path `/api/v1/payments/webhook`. Example: `https://abcd-1234-56ef.ngrok-free.app/api/v1/payments/webhook`
4. Under **Events to listen to**, select `checkout.session.completed` and `payment_intent.payment_failed`.
5. Click **Add endpoint**.
6. On the endpoint detail page, click **Reveal** under **Signing secret** and copy the `whsec_...` value.

Fill in `.env.docker`:

```
STRIPE_API_KEY=sk_test_...
STRIPE_PUBLISHABLE_KEY=pk_test_...
STRIPE_WEBHOOK_SECRET=whsec_...
```

Important

When the ngrok URL changes after a restart, update only the **Endpoint URL** in the Stripe dashboard. The `STRIPE_WEBHOOK_SECRET` stays the same — do *not* regenerate it.

3.6 Step 6 — OpenAI API key

The Safe Job Procedure (SJP) generation feature calls the OpenAI API. `OPENAI_API_KEY` is a required setting — the app refuses to start without it, even if you do not plan to exercise SJP generation yourself.

1. Go to https://platform.openai.com and sign in.
2. Open **API keys**: https://platform.openai.com/api-keys.
3. Click **Create new secret key**, label it `ohs-remote-dev`, and copy the key. You will not see it again.
4. If your account has no free credit, add a small amount at https://platform.openai.com/settings/organization/billing. The default model (`gpt-5-mini`) is inexpensive — a few dollars lasts a long time in development.

Fill in `.env.docker`:

```
OPENAI_API_KEY=sk-...
```

The optional `LLM_*` tuning variables have sensible defaults in `app/config.py` — leave them commented out unless you have a specific reason to change them.

4 Configuration Reference

Required variables, sourced authoritatively from `app/config.py`. The app fails fast at startup if any of these are missing.

Variable	Source	Purpose
`SECRET_KEY`	`openssl rand -hex 32`	JWT / session crypto
`DATABASE_URL`	Pre-filled in `.env.docker.example`	MySQL connection inside Docker network
`SMTP_HOST`	`smtp.gmail.com`	SMTP server
`SMTP_PORT`	`587`	SMTP port (TLS)
`SMTP_USER`	Your Gmail address	SMTP auth
`SMTP_PASSWORD`	Gmail App Password	SMTP auth
`SMTP_FROM_EMAIL`	Same as `SMTP_USER`	From header on outbound email
`STRIPE_API_KEY`	Stripe dashboard → API keys	Payment processing
`STRIPE_PUBLISHABLE_KEY`	Same page	Exposed to frontend for Checkout
`STRIPE_WEBHOOK_SECRET`	Stripe dashboard → Webhooks → your endpoint	Webhook signature verification
`OPENAI_API_KEY`	OpenAI dashboard → API keys	SJP generation
`NGROK_AUTHTOKEN`	ngrok dashboard → Your Authtoken	Public tunnel
`ALLOWED_ORIGINS`	Comma-separated frontend origins	CORS

Optional variables with sensible defaults: `LLM_MODEL`, `LLM_TEMPERATURE`, `LLM_MAX_CONCURRENT_REQUESTS`, `LLM_INPUT_COST_PER_1K_TOKENS`, `LLM_OUTPUT_COST_PER_1K_TOKENS`, `MAX_LOGO_SIZE_MB`, `ALLOWED_LOGO_EXTENSIONS`, `AUTH_OTP_*` (rate limiting for the login OTP flow), `LOG_LEVEL`.

5 Running the Stack

5.1 First run

Once `.env.docker` is fully populated:

```
docker-compose up --build
```

The first build takes several minutes. Docker downloads the MySQL and Python base images, then installs Python dependencies from `requirements.txt`. Subsequent runs start in 10–30 seconds thanks to layer caching.

Three containers come up:

Service	Host port	Purpose
`mysql`	3307	MySQL 8.0, database `ohs_remote_dev`
`app`	8000	FastAPI app, runs `alembic upgrade head` then uvicorn with `-reload`
`ngrok`	4040	Public HTTPS tunnel to `app:8000`, dashboard at localhost:4040

Database migrations run automatically on `app` container startup, so no manual `alembic upgrade` is needed.

5.2 Seed the plans table

After the containers are healthy, run the seed script once:

```
docker-compose exec app python scripts/seed_plans.py
```

This creates the available service plans (Basic, Comprehensive, Industry-Specific). It is idempotent — running it a second time is safe.

5.3 Verification checklist

Confirm the stack is healthy before moving on.

Check	How
All three containers running	`docker-compose ps` shows `mysql`, `app`, `ngrok` all `Up` (mysql healthy)
Health check returns 200	http://localhost:8000/api/v1/health returns `{"status": "healthy"}`

Check	How
Swagger UI loads	http://localhost:8000/docs renders the endpoint list
Migrations applied	`docker-compose exec app alembic current` shows the latest revision
ngrok tunnel active	http://localhost:4040 shows a `https://*.ngrok-free.app` forwarding URL

6 Day-to-day Commands

6.1 Lifecycle

```
docker-compose up              # start, foreground, follow logs
docker-compose up -d           # start in background
docker-compose up --build      # rebuild after requirements.txt change
docker-compose down            # stop and remove containers, keep DB volume
docker-compose down -v         # stop and wipe DB volume --fresh database
docker-compose restart app     # restart just the app
docker-compose ps              # show service status and health
```

6.2 Logs

```
docker-compose logs -f app
docker-compose logs -f mysql
docker-compose logs -f ngrok   # shows the current public URL
docker-compose logs --tail=200 app
```

6.3 Tests, linting, type checking

All tooling runs inside the `app` container so you get the exact pinned dependencies.

```
docker-compose exec app pytest
docker-compose exec app pytest tests/unit/
docker-compose exec app pytest tests/api/
docker-compose exec app pytest --cov=app --cov-report=html
docker-compose exec app ruff check app/
docker-compose exec app ruff check app/ --fix
docker-compose exec app mypy app/
```

6.4 Database migrations

Migrations auto-apply on startup. Run alembic manually only when authoring a new migration.

```
docker-compose exec app alembic current
docker-compose exec app alembic upgrade head
docker-compose exec app alembic downgrade -1
docker-compose exec app alembic revision -m "add foo column to orders"
```

After creating a new revision, edit the generated file under `alembic/versions/` and fill in both `upgrade()` and `downgrade()` bodies before restarting the app.

6.5 Adding a Python dependency

Dependencies are frozen inside the image. After editing `requirements.txt`:

```
docker-compose up --build
```

A plain `docker-compose restart app` is *not* enough — the new package is not in the image yet.

7 Build and Deployment

A production deployment configuration is not currently checked in. The Docker Compose stack in this repository targets local development only. When a production environment is added, it should:

- Disable `-reload` in uvicorn and run with `-workers 4` (or behind a process manager).
- Set `ENVIRONMENT=production`, `DEBUG=false`.
- Source every secret (`SECRET_KEY`, DB credentials, Stripe keys, OpenAI key, SMTP credentials) from a secret manager (AWS Secrets Manager, Google Secret Manager, Azure Key Vault). Never use default values for these in production.
- Point `DATABASE_URL` at a managed MySQL instance (RDS, Cloud SQL, Azure Database for MySQL) with TLS enabled.
- Replace the ngrok tunnel with a real public hostname and update the Stripe webhook endpoint permanently.
- Configure `ALLOWED_ORIGINS` with the production frontend domain(s), not localhost.

The future production Docker image can be built independently of the development Compose file:

```
docker build -t ohs-remote-backend:latest -f docker/Dockerfile .
```

Cloud deployment targets under consideration include AWS ECS + RDS, Google Cloud Run + Cloud SQL, and Azure Container Apps + Azure Database for MySQL. None of these are implemented yet.

8 Troubleshooting

The most common failure modes on this project and how to fix them. Start at the top — the earlier entries cover the things that break most often during first-time setup.

8.1 App refuses to start with a Pydantic `ValidationError`

Symptom. `docker-compose up` brings up MySQL, then the `app` container exits with a traceback ending in something like `ValidationError: N validation errors for Settings`.

Cause. `app/config.py` declares required settings as `Field(...)` with no default. The app fails fast if any of them is missing or still a placeholder like `...` or `sk_test_...`.

Fix. Confirm every variable listed in the Configuration Reference has a real value in `.env.docker`, then re-create the container:

```
docker-compose up -d --force-recreate app
```

A plain `restart` is sometimes not enough — Compose only re-reads `env_file:` on container creation.

8.2 Port already in use

Symptom. `Error: listen tcp 0.0.0.0:8000: bind: address already in use.`

Fix.

```
lsof -i :8000
kill -9 <PID>
```

The offending process is almost always a previous `docker-compose` run that did not shut down cleanly. `docker-compose down` usually resolves it.

8.3 "Can't connect to MySQL server on 'mysql'"

Possible causes.

1. MySQL has not finished initializing. First-run MySQL takes 20–30 seconds. The `app` service has a `depends_on: mysql: condition: service_healthy`, so this should not actually happen — if it does, wait and try again.
2. Wrong `DATABASE_URL` in `.env.docker`. Inside the Docker network the host is `mysql` (the service name), not `localhost`:

   ```
   DATABASE_URL=mysql+pymysql://ohs_dev_user:ohs_dev_password@mysql:3306/ohs_remote_dev
   ```

3. DB volume is in a weird state after a rename. Nuke it:

   ```
   docker-compose down -v
   docker-compose up --build
   ```

   This deletes the MySQL volume. Re-run `python scripts/seed_plans.py` afterwards.

8.4 Alembic "Target database is not up to date" or multiple heads

Fix (multiple heads, typical after a merge).

```
docker-compose exec app alembic heads
docker-compose exec app alembic merge -m "merge heads" <head_a> <head_b>
docker-compose exec app alembic upgrade head
```

Fix (fresh DB, if you have no local data to preserve).

```
docker-compose down -v
docker-compose up --build
docker-compose exec app python scripts/seed_plans.py
```

8.5 Gmail rejects SMTP authentication

Symptom. `535-5.7.8 Username and Password not accepted` or `Application-specific password required`.

Fix.

1. Enable 2-Step Verification on your Google account.
2. Generate an App Password at https://myaccount.google.com/apppasswords.
3. Paste into `SMTP_PASSWORD`. Spaces are optional.
4. Confirm `SMTP_FROM_EMAIL` matches `SMTP_USER`.
5. `docker-compose up -d -force-recreate app`.

8.6 Stripe webhook never fires

Symptom. A successful Stripe Checkout completes but the order stays in `draft` / `pending`, no document is generated, no email is sent.

Checks, in order.

1. Is ngrok running? Open http://localhost:4040 — you should see a live forwarding URL. If blank, check `docker-compose logs -f ngrok` for an auth error.
2. Is the Stripe webhook pointing at the *current* ngrok URL? It rotates on every restart. Update at https://dashboard.stripe.com/test/webhooks. The URL must end with `/api/v1/payments/webhook`.
3. Is the webhook signing secret correct? Confirm it matches `STRIPE_WEBHOOK_SECRET`. This value does not change when you update the endpoint URL — do not regenerate it.
4. Check Stripe's delivery log on the webhook endpoint page. A `400` with "signature verification failed" means the signing secret is wrong; a connection error means ngrok is not forwarding.

8.7 OpenAI / SJP generation errors

Symptom. Triggering SJP generation returns 500, or the job transitions to `failed`. Logs show an error from the `openai` SDK.

Common causes.

- `Invalid API key` — `OPENAI_API_KEY` is wrong or unset. Must start with `sk-`.
- `insufficient_quota` — OpenAI account has no credit. Add some at https://platform.openai.com/settings/organization/billing.
- `model not found` — someone pinned `LLM_MODEL` to a model your key cannot access. Remove the override.
- Timeouts / transient network errors — retry. If persistent, check `LLM_MAX_CONCURRENT_REQUESTS`.

8.8 Code changes not taking effect

1. `docker-compose logs -f app` should print `WatchFiles detected changes` when you save.

2. Only `app/`, `alembic/`, `scripts/`, `data/`, and `templates/` are bind-mounted. Files outside those paths require a rebuild.
3. If you edited `requirements.txt`: `docker-compose up -build`.
4. Force a clean restart: `docker-compose restart app`.

8.9 `ModuleNotFoundError` on imports

Usually a teammate added a dependency and you haven't rebuilt. `docker-compose up -build`.

8.10 File permission errors writing to `data/`

Document generation errors with `PermissionError: [Errno 13]` when saving a logo or DOCX.

```
mkdir -p data/uploads/logos data/documents/generated data/documents/previews
chmod -R u+rwX data/
```

8.11 Out of disk space

```
docker system df
docker system prune
docker system prune -a  # more aggressive
```

Do not run `docker volume prune` unless you are prepared to lose your MySQL data volume.

8.12 Nothing works — start over

The nuclear option. Wipes the DB, rebuilds everything cleanly.

```
docker-compose down -v
docker system prune -f
docker-compose up --build
docker-compose exec app python scripts/seed_plans.py
```

If this does not get the stack to a healthy state, the problem is in your environment (Docker Desktop not running, `.env.docker` malformed, etc.). Collect `docker-compose logs -tail=200 app` and `docker-compose ps` output before asking for help.

9 Where to Go Next

After the stack is healthy:

- Explore the live API at `http://localhost:8000/docs` (Swagger UI).
- Read the repo's root `README.md` for the documentation map.
- Read `docs/ARCHITECTURE.md` for the layered architecture and design patterns.
- Read `docs/API_OVERVIEW.md` for the full order flow and endpoint map.
- Read `docs/ADMIN_API.md` for the admin-side endpoints.

- Read `docs/SJP_FRONTEND_GUIDE.md` for the async SJP generation flow.
- Read `docs/PYTHON_BACKEND_CONCEPTS.md` if you are new to FastAPI / Pydantic / SQLAlchemy.
- Read `CLAUDE.md` for the project-wide coding standards.